

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении лабораторной работы №3
по дисциплине «Методы тестирования программного
обеспечения»
«Статический анализ кода»

Выполнила студентка группы
№5130201/30101
Проверил

Лаишевкина А. М. _____
Курочкин М. А. _____

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	4
2 Описание тестируемой программы	5
3 Статический анализ кода	6
3.1 Методы статического анализа кода	6
3.1.1 CppCheck	6
3.1.2 Pylint	6
3.1.3 SpotBugs	7
3.1.4 Сравнение инструментов	7
4 Описание CppCheck	8
5 Анализ программы	11
5.1 Запуск CppCheck	11
5.2 Анализ и исправление недочетов	12
Заключение	14
Список литературы	15
Приложение А. Исходный код программы	16
Приложение В. Исправленный код программы	20

Введение

В данной лабораторной работе представлено описание использования методов статического анализа кода.

Статический анализ кода – анализ исходного кода программного обеспечения, производимый без реального выполнения программ.

Sprcheck – статический анализатор кода для языков C и C++, предназначенный для поиска проблем, которые не обнаруживаются компиляторами.

1 Постановка задачи

При выполнении данной лабораторной работы требуется изучить методы статического анализа кода и провести статическое тестирование программы с помощью CppCheck.

2 Описание тестируемой программы

Название программы: Проверка графа на эйлеровость и построение эйлерова цикла.

Дано:

- n – количество вершин графа;
- A – матрица смежности неориентированного графа размера $n \times n$.

Требуется: Выдать результат проверки неориентированного графа на эйлеровость и вывести эйлеров цикл, если таковой имеется.

Ограничения:

- $0 \leq n \leq 100$, n – целое число;
- граф неориентированный (матрица симметрична относительно главной диагонали).

Таблица 1. Спецификация программы

№	Входные данные	Результат	Реакция программы
1	$n = 0$	Окончание работы программы	Завершает работу
2	$n = 1$ или $n = 2$	Граф не эйлеров	Вывод: «Граф НЕ является эйлеровым».
3	$n \geq 3$, связный, все степени чётные	Граф эйлеров	Вывод: «Граф является эйлеровым!» + цикл.
4	$n \geq 3$, не связный, все степени четные	Граф не эйлеров	Вывод: «Граф НЕ является эйлеровым».
5	$n = \text{"abc"}$	Ошибка ввода	Вывод: «Ошибка: ожидалось целое число.» + повтор запроса.
6	$n = 3$, $A = [[0, 1, 'b'], [1, 0, 1], ['b', 1, 0]]$	Ошибка ввода	Вывод: «Ошибка: ожидалось целое число.» + повтор запроса.

Исходный код программы приведен в [Приложении А](#).

3 Статический анализ кода

Статический анализ – один из вариантов проверки исходного кода ПО. Обычно его используют, чтобы найти самые популярные ошибки и уязвимости на ранних этапах разработки.

Проводить такую проверку можно вручную, но гораздо проще и быстрее задачу выполняют автоматизированные SAST-инструменты. Они сканируют синтаксис кода и сравнивают его с набором шаблонов и правил, которые помогают обнаружить уязвимости и другие проблемы в коде. Статический анализатор кода получает написанный разработчиком код, исследует его и выдает отчет с ошибками и уязвимостями.

Статический анализ также может включать проверку стиля кода, что помогает поддерживать единообразие в проекте. Это особенно важно в командах, где несколько разработчиков работают над одним проектом. Инструменты для статического анализа могут быть настроены для проверки соответствия кода определенным стандартам, что помогает избежать распространенных проблем и улучшить читаемость кода.

Многие инструменты для статического анализа могут быть интегрированы в процесс сборки, что позволяет автоматизировать проверку кода. Это обеспечивает постоянный контроль качества и помогает разработчикам сосредоточиться на написании кода, а не на его проверке.

3.1 Методы статического анализа кода

3.1.1 CppCheck

Cppcheck – это инструмент статического анализа кода для языков C и C++, предназначенный для поиска распространенных ошибок программирования без запуска программы. Он способен обнаруживать утечки памяти, использование неинициализированных переменных, выход за границы массивов и другие потенциально опасные конструкции. Главными преимуществами Cppcheck являются высокая скорость работы, простота использования и небольшое количество ложных предупреждений. Данный инструмент часто применяется в учебных проектах и при быстрой проверке исходного кода.

3.1.2 Pylint

Pylint предназначен для статического анализа программ на языке Python. Он проверяет код на наличие ошибок, нарушений стиля оформления и потенциально опасных конструкций. Кроме поиска синтаксических и логических проблем, Pylint оценивает качество структуры программы и может выдавать итоговую оценку кода. Инструмент помогает поддерживать единый стиль программирования и часто используется совместно с другими средствами анализа Python-кода.

3.1.3 SpotBugs

SpotBugs – инструмент статического анализа для Java, предназначенный для обнаружения ошибок и подозрительных конструкций в байт-коде программ. Он способен находить проблемы, связанные с многопоточностью, обработкой исключений, работой с памятью и безопасностью. Анализ выполняется уже после компиляции программы, что позволяет выявлять ошибки, которые трудно заметить при обычной проверке исходного кода. SpotBugs активно применяется в Java-проектах для повышения надёжности программного обеспечения.

3.1.4 Сравнение инструментов

Несмотря на то что CppCheck, Pylint и SpotBugs предназначены для разных языков, они решают схожие задачи. Все три инструмента позволяют обнаруживать недочёты без запуска программы, что существенно ускоряет цикл разработки. При этом каждый из них имеет свою специфику: CppCheck работает непосредственно с исходным кодом и отличается малым количеством ложных срабатываний, Pylint помимо поиска недочётов оценивает качество структуры и стиль оформления кода, а SpotBugs анализирует уже скомпилированный байт-код, что позволяет ему выявлять недочёты, связанные с многопоточностью и безопасностью.

Основным ограничением статического анализа в целом является невозможность выявить недочёты, зависящие от конкретных входных данных или поведения среды выполнения – для их обнаружения необходимо динамическое тестирование.

4 Описание CppCheck

CppCheck – статический анализатор кода для языка C/C++, предназначенный для поиска ошибок, которые не обнаруживаются компиляторами.

Категории проверок

- `error` – явные ошибки, которые анализатор считает критическими и обычно они приводят к багам;
- `warning` – предупреждения, здесь даются сообщения о небезопасном коде;
- `style` – стилистические недочеты, сообщения появляются в случае неаккуратного кодирования;
- `performance` – проблемы производительности, CppCheck предлагает варианты, как сделать код быстрее;
- `portability` – ошибки совместимости, обычно связано с различным поведением компиляторов или систем разной разрядности;
- `information` – информационные сообщения, возникающие в зоде проверки и не связаны с ошибками в коде;
- `unusedFunction` – попытка вычислить неиспользуемые функции, не умеет работать в многопоточном режиме;
- `missingInclude` – проверка на недостающие `include`.

Проверки ключаются параметром – `enable`, список категорий проверок перечисляется через запятую. Можно использовать ключевое слово `all`, которое включает все перечисленные проверки.

Аргументы командной строки:

- `-j` – параметр, позволяющий запускать проверку в многопоточном режиме. В качестве параметра передаётся количество процессоров.
- `-q` – тихий режим. По умолчанию `cppcheck` выдаёт информационные сообщения о ходе проверки. Данный параметр полностью выключает информационные сообщения, остаются только сообщения об ошибках.
- `-f` или `-force` – включить перебор всех вариантов директив `ifdef`.
- `-v` – режим отладки — `cppcheck` выдаёт внутреннюю информацию о ходе проверки.
- `-xml` – выводит результат проверки в формате XML.
- `template=gcc` – выводит ошибки в формате предупреждений компилятора `gcc`.
- `supress` – режим подавления ошибок с указанными идентификаторами.
- `-h` – выдаёт справку по всем параметрам на английском языке.

Правила проверки

- 64-bit portability;
- Assert;
- Auto Variables;
- Boolean;
- Boost usage;
- Bounds checking;
- Check function usage;
- Class;
- Condition;
- Exception Safety;
- IO using format string;
- Leaks (auto variables);
- Memory leaks (address not taken);
- Memory leaks (class variables);
- Memory leaks (function variables);
- Memory leaks (struct members);
- Null pointer;
- Other;
- STL usage;
- Sizeof;
- String;
- Type;
- Uninitialized variables;
- Unused functions;
- UnusedVar;
- Using postfix operators;
- Vaarg.

Разбор CppCheck правил из группы «Check function usage»

CppCheck предоставляет проверки из группы «Check function usage» для обнаружения некорректного использования функций. Рассмотрим следующие правила:

- **Rule: missingReturn**

Severity: Error

Данное правило обнаруживает ситуации, когда функция с типом возврата, отличным от `void`, имеет пути выполнения, на которых отсутствует оператор `return`. Это приводит к неопределенному поведению программы.

- **Rule: ignoredReturnValue**

Severity: Warning

Проверка предупреждает, когда возвращаемое значение функции не используется. Это особенно важно для функций, которые возвращают коды ошибок или критически важные результаты, которые должны быть проверены.

Разбор CppCheck правил из группы «Check function usage»

CppCheck предоставляет проверки из группы «Null pointer» для обнаружения ошибок, связанных с нулевыми указателями. Рассмотрим следующие правила:

- **Rule: nullPointer**

Severity: Error / Warning

Данное правило обнаруживает разыменование нулевого указателя. Это происходит, когда код пытается получить доступ к данным через указатель, который имеет значение `NULL`/`nullptr`. Такие ошибки приводят к неопределенному поведению и чаще всего к аварийному завершению программы.

- **Rule: nullPointerArithmetic**

Severity: Error / Warning

Проверка обнаруживает арифметические операции с нулевым указателем. Выполнение арифметики (сложение, вычитание, инкремент, декремент) над нулевым указателем приводит к неопределенному поведению согласно стандарту C/C++.

5 Анализ программы

5.1 Запуск CppCheck

Для проведения статического анализа кода программы необходимо:

1. Скачать и распаковать CppCheck.
2. Открыть командную строку (Win + R, ввести cmd).
3. Перейти в директорию с cppcheck.exe:
cd путь_к_директории
4. Выполнить анализ:
cppcheck -enable=all -inconclusive путь_к_исходным_файлам

Статический анализ выполнялся со следующими параметрами:

- -enable=all – активация всех возможных категорий проверок;
- -inconclusive – включение в вывод предупреждений, которые алгоритм не может однозначно подтвердить;
- “D:/university/6 semester/testing/3 lab/prog.cpp” – путь к тестируемой программе.

При выполнении анализа CppCheck было активировано 183 проверки. В результате запуска статического анализатора Cppcheck были выявлены следующие недочеты:

```
1 D:\Program Files\cppcheck> cppcheck --enable=all --inconclusive "D:/
  university/6 semester/testing/3 lab/prog.cpp"
2 Checking prog.cpp ...
3 prog.cpp:1:2: information: Include file: <iostream> not found. Please note:
  Standard library headers do not need to be provided to get proper results
  . [missingIncludeSystem]
4 #include <iostream>
5 ^
6 prog.cpp:2:2: information: Include file: <vector> not found. Please note:
  Standard library headers do not need to be provided to get proper results
  . [missingIncludeSystem]
7 #include <vector>
8 ^
9 prog.cpp:3:2: information: Include file: <random> not found. Please note:
  Standard library headers do not need to be provided to get proper results
  . [missingIncludeSystem]
10 #include <random>
11 ^
12 prog.cpp:4:2: information: Include file: <queue> not found. Please note:
  Standard library headers do not need to be provided to get proper results
  . [missingIncludeSystem]
13 #include <queue>
14 ^
15 prog.cpp:5:2: information: Include file: <stack> not found. Please note:
  Standard library headers do not need to be provided to get proper results
  . [missingIncludeSystem]
```

```

16 #include <stack>
17 ^
18 prog.cpp:6:2: information: Include file: <limits> not found. Please note:
    Standard library headers do not need to be provided to get proper results
    . [missingIncludeSystem]
19 #include <limits>
20 ^
21 prog.cpp:0:0: information: Limiting analysis of branches. Use --check-level=
    exhaustive to analyze all branches. [normalCheckLevelMaxBranches]
22 ^
23 prog.cpp:62:10: style: inconclusive: Technically the member function 'Graph
    ::printEulerCycle' can be const. [functionConst]
24     void printEulerCycle() {
25         ^
26 prog.cpp:17:38: performance: Function parameter 'am' should be passed by
    const reference. [passedByValue]
27     Graph(int v, vector<vector<int>> am) : vertices(v), adjacencyMatrix(am)
        {}
28         ^
29 prog.cpp:55:17: style: Consider using std::count_if algorithm instead of a
    raw loop. [useStlAlgorithm]
30         oddDegreeCount++;
31         ^
32 prog.cpp:72:59: style: Consider using std::accumulate algorithm instead of a
    raw loop. [useStlAlgorithm]
33         for (int neighbor : tempMatrix[start]) degree += neighbor;
34             ^
35 nofile:0:0: information: Active checkers: 183/186 (use --checkers-report=<
    filename> to see details) [checkersReport]

```

5.2 Анализ и исправление недочетов

Недочет: Function parameter passed by value

Параметр функции `am` передается по значению, что приводит к ненужному копированию вектора. Следует передавать по `const reference`.

Исправление:

```

1 //до исправления
2 Graph(int v, vector<vector<int>> am) : vertices(v), adjacencyMatrix(am) {}
3
4 //после исправления
5 Graph(int v, const vector<vector<int>>& am) : vertices(v), adjacencyMatrix(
    am) {}

```

Недочет: Function can be const

Метод `printEulerCycle` не изменяет состояние объекта и может быть объявлен как `const`.

Исправление:

```

1 //до исправления
2 void printEulerCycle() {
3     if (vertices == 0) return;
4
5 //после исправления
6 void printEulerCycle() const {
7     if (vertices == 0) return;

```

Недочет: Consider using `std::count_if` algorithm

Вместо ручного цикла для подсчета элементов с нечетной степенью рекомендуется использовать алгоритм `std::count_if`.

Исправление:

```
1 //до исправления
2 int oddDegreeCount = 0;
3 for (int d : degree) {
4     if (d % 2 != 0) {
5         oddDegreeCount++;
6     }
7 }
8
9 //после исправления
10 #include <algorithm>
11 int oddDegreeCount = std::count_if(degree.begin(), degree.end(),
12                                   [](int d){ return d % 2 != 0; });
```

Недочет: Consider using `std::accumulate` algorithm

Вместо ручного цикла для суммирования значений рекомендуется использовать алгоритм `std::accumulate`.

Исправление:

```
1 //до исправления
2 int start = 0;
3 for (; start < vertices; ++start) {
4     int degree = 0;
5     for (int neighbor : tempMatrix[start]) degree += neighbor;
6     if (degree > 0) break;
7 }
8
9 //после исправления
10 #include <numeric>
11 int start = 0;
12 for (; start < vertices; ++start) {
13     int degree = std::accumulate(tempMatrix[start].begin(),
14                                   tempMatrix[start].end(), 0);
15     if (degree > 0) break;
16 }
```

Недочет: Missing include files (information)

СppCheck не находит системные заголовочные файлы. Это информационное сообщение, не требующее исправления кода, так как заголовочные файлы в коде присутствуют.

Исправленный код программы приведен в [Приложении В](#).

Заключение

В ходе выполнения лабораторной работы был проведен статический анализ кода при помощи CppCheck. Использование CppCheck позволило обнаружить недочеты в коде программы. Их исправление помогло мне больше узнать о написании чистого и понятного кода. Я сделала вывод, что статическое тестирование, в частности CppCheck, не только помогает находить недочеты в коде, но и предлагает улучшения для читаемости кода.

Список литературы

1. Майерс, Г. Искусство тестирования программ / Г. Майерс, Т. Баджетт, К. Сандлер. - Изд. 3-е. - Санкт-Петербург: Диалектика, 2012 - 272 с.
2. Cppcheck manual (Электронный ресурс).- URL: <http://cppcheck.net/manual.pdf>
(дата обращения: 02.04.2026)

Приложение А. Исходный код программы

```
1 #include <iostream>
2 #include <vector>
3 #include <random>
4 #include <queue>
5 #include <stack>
6 #include <limits>
7
8 using namespace std;
9
10 class Graph
11 {
12 private:
13     int vertices;
14     vector<vector<int>> adjacencyMatrix;
15
16 public:
17     Graph(int v, vector<vector<int>> am) : vertices(v), adjacencyMatrix(am)
18     {}
19
20     bool isEulerian(){
21         if (vertices <= 2) {
22             return false;
23         }
24
25         vector<bool> visited(vertices, false);
26         queue<int> q;
27         q.push(0);
28         visited[0] = true;
29         int visitedCount = 1;
30
31         while (!q.empty()) {
32             int u = q.front();
33             q.pop();
34             for (int v = 0; v < vertices; ++v) {
35                 if (adjacencyMatrix[u][v] && !visited[v]) {
36                     visited[v] = true;
37                     q.push(v);
38                     visitedCount++;
39                 }
40             }
41             if (visitedCount != vertices) {
42                 return false;
43             }
44
45             vector<int> degree(vertices, 0);
46             for (int u = 0; u < vertices; ++u) {
47                 for (int v = 0; v < vertices; ++v) {
48                     degree[u] += adjacencyMatrix[u][v];
49                 }
50             }
51
52             int oddDegreeCount = 0;
53             for (int d : degree) {
54                 if (d % 2 != 0) {
55                     oddDegreeCount++;
56                 }
57             }
```



```

58         return (oddDegreeCount == 0 || oddDegreeCount == 2);
59     }
60
61     void printEulerCycle() {
62         if (vertices == 0) return;
63
64         vector<vector<int>> tempMatrix = adjacencyMatrix;
65         stack<int> stack;
66         vector<int> cycle;
67
68         int start = 0;
69         for (; start < vertices; ++start) {
70             int degree = 0;
71             for (int neighbor : tempMatrix[start]) degree += neighbor;
72             if (degree > 0) break;
73         }
74
75         stack.push(start);
76
77         while (!stack.empty()) {
78             int current = stack.top();
79
80             bool found = false;
81             for (int next = 0; next < vertices; ++next) {
82                 if (tempMatrix[current][next] > 0) {
83                     tempMatrix[current][next]--;
84                     tempMatrix[next][current]--;
85                     stack.push(next);
86                     found = true;
87                     break;
88                 }
89             }
90
91             if (!found) {
92                 cycle.push_back(current);
93                 stack.pop();
94             }
95         }
96
97         cout << "Эйлеров цикл: ";
98         for (size_t i = 0; i < cycle.size(); ++i) {
99             cout << cycle[i];
100             if (i != cycle.size() - 1) cout << " -> ";
101         }
102         cout << " -> " << cycle[0] << endl;
103     }
104 };
105
106 vector<vector<int>> generateAdjacencyMatrix(int n) // generates unoriented
107     adjacency matrix
108 {
109     random_device rd;
110     mt19937 gen(rd());
111     uniform_int_distribution<> distrib(0, 1);
112
113     vector<vector<int>> matrix(n, vector<int>(n, 0));
114
115     for (int i = 0; i < n; ++i) {
116         for (int j = i + 1; j < n; ++j) {

```

```

117         int val = distrib(gen);
118         matrix[i][j] = val;
119         matrix[j][i] = val;
120     }
121 }
122 return matrix;
123 }
124
125 vector<vector<int>> inputAdjacencyMatrix(int n) {
126     vector<vector<int>> matrix(n, vector<int>(n));
127     cout << "Введите матрицу смежности (" << n << "x" << n << "):\n";
128     for (int i = 0; i < n; ++i) {
129         for (int j = 0; j < n; ++j) {
130             if (!(cin >> matrix[i][j])) {
131                 cerr << "Ошибка: ожидалось целое число.\n";
132                 cin.clear();
133                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
134                 return {};
135             }
136         }
137     }
138     cin.ignore(numeric_limits<streamsize>::max(), '\n');
139     return matrix;
140 }
141
142 int main() {
143     while (true) {
144         int vertices;
145         cout << "Введите количество вершин (0 для выхода): ";
146         cin >> vertices;
147         cin.ignore(numeric_limits<streamsize>::max(), '\n');
148
149         if (vertices == 0) {
150             break;
151         }
152
153         char choice;
154         cout << "Сгенерировать матрицу смежности автоматически? (y/n): ";
155         cin >> choice;
156         cin.ignore(numeric_limits<streamsize>::max(), '\n');
157
158         vector<vector<int>> adjMatrix;
159         if (choice == 'y' || choice == 'Y') {
160             adjMatrix = generateAdjacencyMatrix(vertices);
161             cout << "Сгенерированная матрица:\n";
162             for (const auto& row : adjMatrix) {
163                 for (int val : row) {
164                     cout << val << " ";
165                 }
166                 cout << "\n";
167             }
168         } else {
169             adjMatrix = inputAdjacencyMatrix(vertices);
170         }
171
172         Graph graph(vertices, adjMatrix);
173
174         if (graph.isEulerian()) {
175             cout << "Граф является эйлеровым!\n";
176             graph.printEulerCycle();

```

```
177         } else {
178             cout << "Граф НЕ является эйлеровым.\n";
179         }
180         cout << "\n";
181     }
182     return 0;
183 }
```

Приложение В. Исправленный код программы

```
1 #include <iostream>
2 #include <vector>
3 #include <random>
4 #include <queue>
5 #include <stack>
6 #include <limits>
7 #include <algorithm> // для std::count_if
8 #include <numeric>    // для std::accumulate
9
10 using namespace std;
11
12 class Graph
13 {
14 private:
15     int vertices;
16     vector<vector<int>> adjacencyMatrix;
17
18 public:
19     // Исправление 1: передача параметра по const reference
20     Graph(int v, const vector<vector<int>>& am) : vertices(v),
21         adjacencyMatrix(am) {}
22
23     bool isEulerian(){
24         if (vertices <= 2) {
25             return false;
26         }
27
28         vector<bool> visited(vertices, false);
29         queue<int> q;
30         q.push(0);
31         visited[0] = true;
32         int visitedCount = 1;
33
34         while (!q.empty()) {
35             int u = q.front();
36             q.pop();
37             for (int v = 0; v < vertices; ++v) {
38                 if (adjacencyMatrix[u][v] && !visited[v]) {
39                     visited[v] = true;
40                     q.push(v);
41                     visitedCount++;
42                 }
43             }
44             if (visitedCount != vertices) {
45                 return false;
46             }
47
48             vector<int> degree(vertices, 0);
49             for (int u = 0; u < vertices; ++u) {
50                 for (int v = 0; v < vertices; ++v) {
51                     degree[u] += adjacencyMatrix[u][v];
52                 }
53             }
54
55             // Исправление 2: использование std::count_if вместо цикла
56             int oddDegreeCount = std::count_if(degree.begin(), degree.end(),
57                 [](int d){ return d % 2 != 0; });
```

```

58     return (oddDegreeCount == 0 || oddDegreeCount == 2);
59 }
60
61 // Исправление 3: добавлен const для метода
62 void printEulerCycle() const {
63     if (vertices == 0) return;
64
65     vector<vector<int>> tempMatrix = adjacencyMatrix;
66     stack<int> stack;
67     vector<int> cycle;
68
69     int start = 0;
70     for (; start < vertices; ++start) {
71         // Исправление 4: использование std::accumulate вместо цикла
72         int degree = std::accumulate(tempMatrix[start].begin(),
73                                     tempMatrix[start].end(), 0);
74         if (degree > 0) break;
75     }
76
77     stack.push(start);
78
79     while (!stack.empty()) {
80         int current = stack.top();
81
82         bool found = false;
83         for (int next = 0; next < vertices; ++next) {
84             if (tempMatrix[current][next] > 0) {
85                 tempMatrix[current][next]--;
86                 tempMatrix[next][current]--;
87                 stack.push(next);
88                 found = true;
89                 break;
90             }
91         }
92
93         if (!found) {
94             cycle.push_back(current);
95             stack.pop();
96         }
97     }
98
99     cout << "Эйлеров цикл: ";
100     for (size_t i = 0; i < cycle.size(); ++i) {
101         cout << cycle[i];
102         if (i != cycle.size() - 1) cout << " -> ";
103     }
104     cout << " -> " << cycle[0] << endl;
105 }
106 };
107
108 vector<vector<int>> generateAdjacencyMatrix(int n) // generates unoriented
109     adjacency matrix
110 {
111     random_device rd;
112     mt19937 gen(rd());
113     uniform_int_distribution<> distrib(0, 1);
114
115     vector<vector<int>> matrix(n, vector<int>(n, 0));
116

```

```

117     for (int i = 0; i < n; ++i) {
118         for (int j = i + 1; j < n; ++j) {
119             int val = distrib(gen);
120             matrix[i][j] = val;
121             matrix[j][i] = val;
122         }
123     }
124     return matrix;
125 }
126
127 vector<vector<int>> inputAdjacencyMatrix(int n) {
128     vector<vector<int>> matrix(n, vector<int>(n));
129     cout << "Введите матрицу смежности (" << n << "x" << n << "):\n";
130     for (int i = 0; i < n; ++i) {
131         for (int j = 0; j < n; ++j) {
132             if (!(cin >> matrix[i][j])) {
133                 cerr << "Ошибка: ожидалось целое число.\n";
134                 cin.clear();
135                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
136                 return {};
137             }
138         }
139     }
140     cin.ignore(numeric_limits<streamsize>::max(), '\n');
141     return matrix;
142 }
143
144 int main() {
145     while (true) {
146         int vertices;
147         cout << "Введите количество вершин (0 для выхода): ";
148         cin >> vertices;
149         cin.ignore(numeric_limits<streamsize>::max(), '\n');
150
151         if (vertices == 0) {
152             break;
153         }
154
155         char choice;
156         cout << "Сгенерировать матрицу смежности автоматически? (y/n): ";
157         cin >> choice;
158         cin.ignore(numeric_limits<streamsize>::max(), '\n');
159
160         vector<vector<int>> adjMatrix;
161         if (choice == 'y' || choice == 'Y') {
162             adjMatrix = generateAdjacencyMatrix(vertices);
163             cout << "Сгенерированная матрица:\n";
164             for (const auto& row : adjMatrix) {
165                 for (int val : row) {
166                     cout << val << " ";
167                 }
168                 cout << "\n";
169             }
170         } else {
171             adjMatrix = inputAdjacencyMatrix(vertices);
172         }
173
174         Graph graph(vertices, adjMatrix);
175
176         if (graph.isEulerian()) {

```

```
177         cout << "Граф является эйлеровым!\n";
178         graph.printEulerCycle();
179     } else {
180         cout << "Граф НЕ является эйлеровым.\n";
181     }
182     cout << "\n";
183 }
184 return 0;
185 }
```